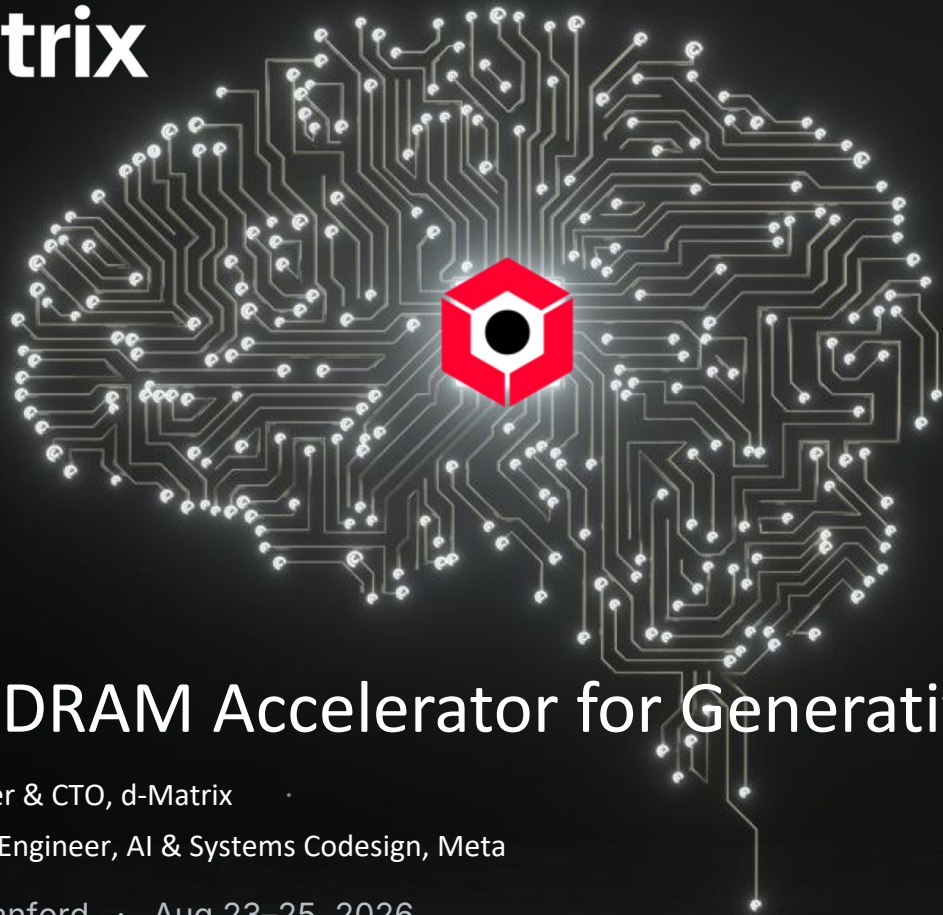




d-Matrix



Raptor:

The First 3D-DRAM Accelerator for Generative Inference

Sudeep Bhoja, Co-founder & CTO, d-Matrix

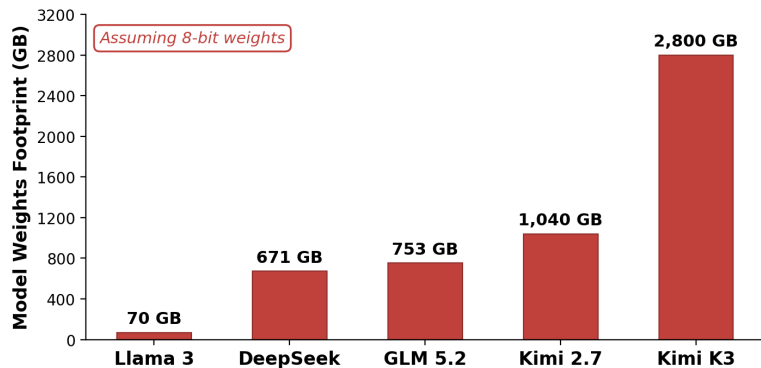
Aayush Ankit,⁺ Software Engineer, AI & Systems Codesign, Meta

Hot Chips 2026 · Stanford · Aug 23–25, 2026

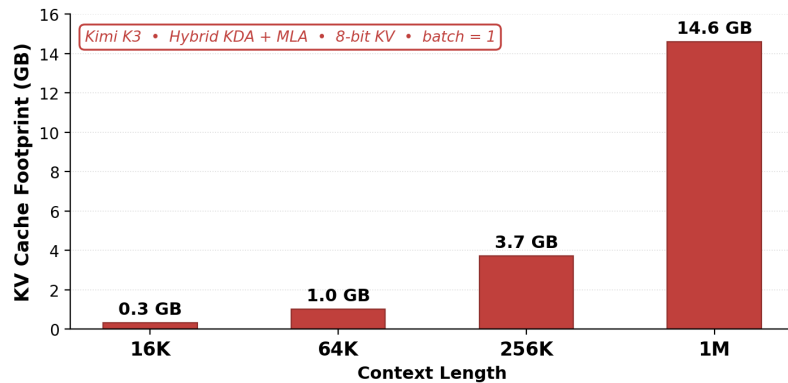
⁺Work done while at d-Matrix

The Growing Data Problem

Model weights keep growing



KV cache grows with context



x batch size
64 users at 1M
context:
~ 935GB KV

Longer context × more users ⇒ Large KV cache

Weights and KV cache present a **capacity and bandwidth problem**. Both are growing.

Today's Memory Technologies

SRAM: Bandwidth Advantage

Why SRAM is fast:

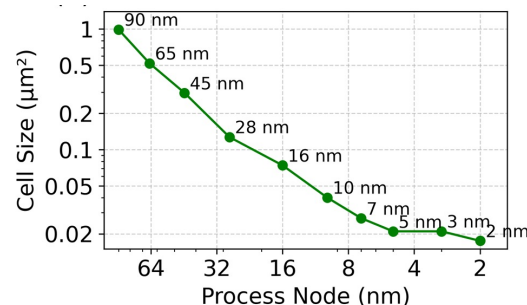
- 6T cell, direct bit-line access
- On-die: no interposer, no package crossing
- Sub-ns access latency

Corsair SRAM Accelerator Specs – card pair:

Metric	Value
Bandwidth	300 TB/s
Capacity	4 GB
Latency	~1 ns
I/O energy	~0.5 pJ/bit

Why SRAM cannot scale:

- 6T cell is **10× larger** than DRAM 1T1C
- Bitcell frozen ~**0.021 μm^2** across **N5 → N3E → N2** (TSMC IEDM/ISSCC)
- Leakage: tens of watts at GB scale
- Practical limit: ~**4 GB per card pair**
- Cost: **100x** DRAM



Source: [Dayo et al.](#)

SRAM is the fastest memory in silicon.

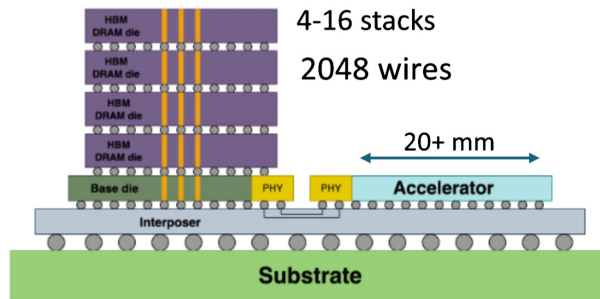
Example use case: Draft model in frontier-LLM speculative decoding.

HBM: The Bandwidth Issue

HBM provides high capacity

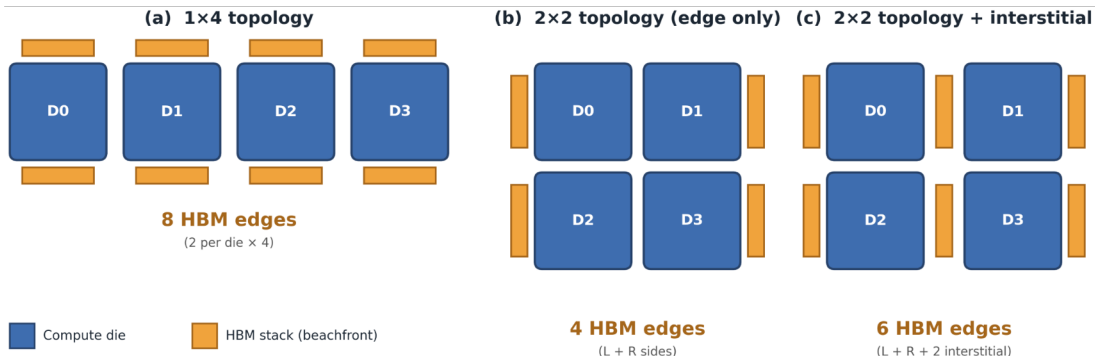
- 1TIC cell, high array density
- 8–16 core dies per stack
- 8 stacks per package

HBM-4

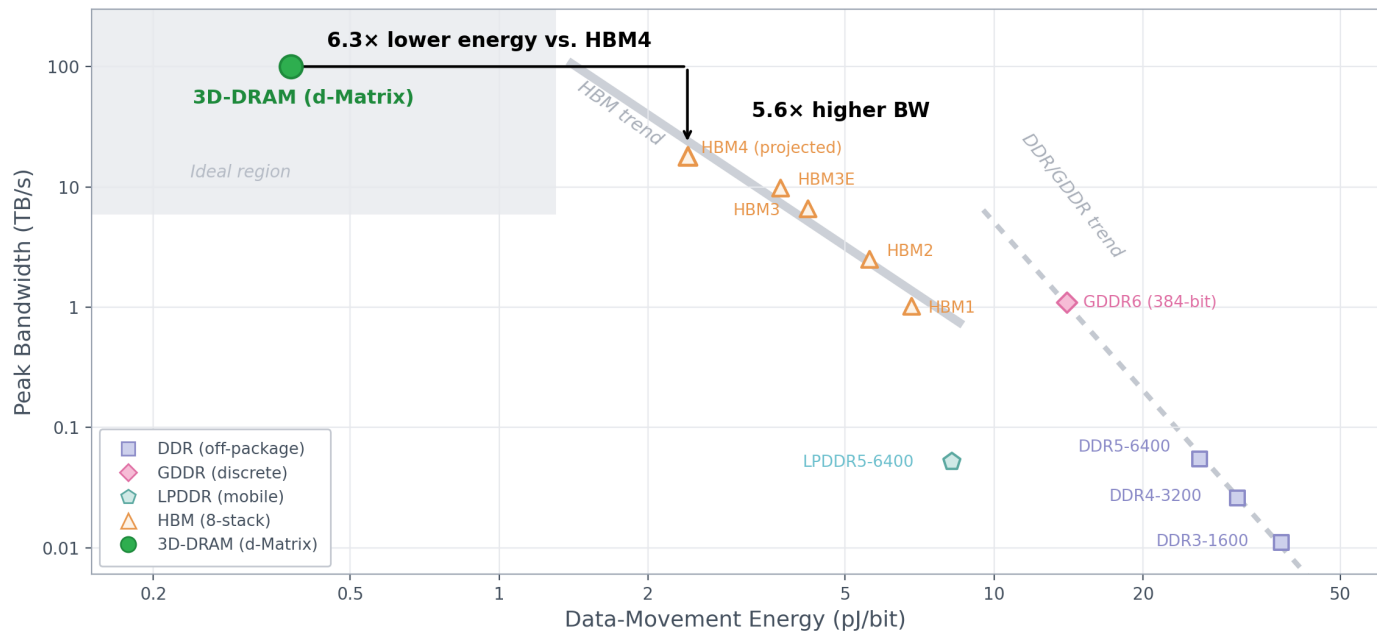


HBM BW is challenging to scale to SRAM-like numbers

- Pin speed, IO width (per HBM base die) have improved slowly
- Number of stacks is beachfront limited
- Dedicating the entire die beachfront to HBM forces harder packaging (interposer size, warpage). Examples of 4 die package beachfronts
- A Practical limit w/ HBM4 (Vera Rubin, MI 455): ~20TB/s



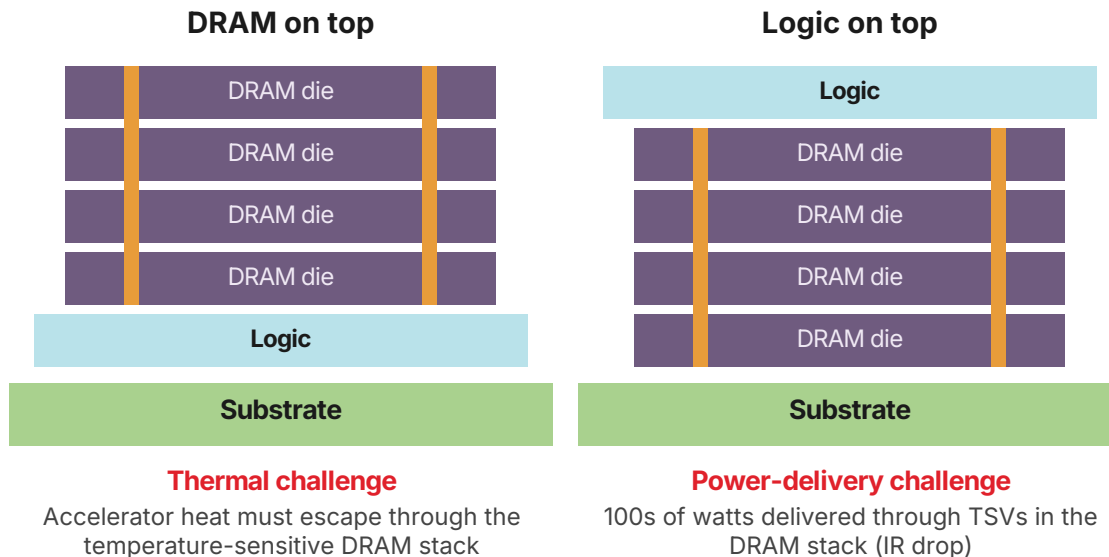
HBM: The Power Problem



At 2.4pJ/bit, 100TB/s
requires 1.92KW for HBM
power only (excluding
fabric power)

HBM capacity is very high but SOTA packages neither have beachfront, nor power for SRAM-like bandwidth.

3D-DRAM: An idea whose time has come

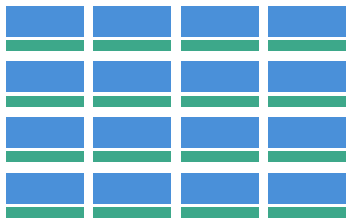


1-Hi stack, Logic-on-Top with power density $\leq 0.5 \text{ W/mm}^2$ can be reliably liquid-cooled (DRAM $< 100^\circ\text{C}$) with required power delivery.

Why 3D-DRAM?

SRAM

On-die memory



Compute + SRAM per tile

Capacity



Low

Bandwidth



High

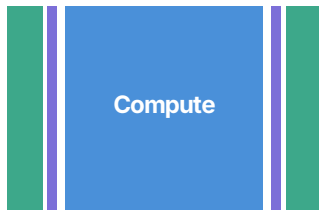
Power



Low

HBM

2D package + beachfront



NoC feeds edge HBM stacks

Capacity



High

Bandwidth



Low

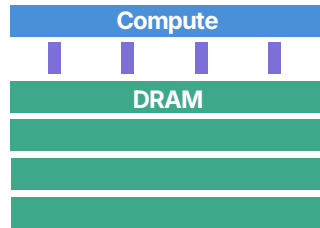
Power



High

3D DRAM

Stacked compute-on-memory



Compute on DRAM, vertical IO

Capacity



Medium

Bandwidth



High

Power



Low-Med

The energy ladder

Memories & interconnect	Energy / bit
SRAM (on-die)	~50 fJ
On-chip wire	~35 fJ/mm
3D vertical IO	0.3–0.4 pJ
Interposer trace	~500 fJ/mm
2.5D HBM4 (system)	2.5pJ + 3pJ (on chip)
3D IO lands ~10× below HBM — PHY-less, mm-scale vertical path instead of cm-scale interposer + PHY	

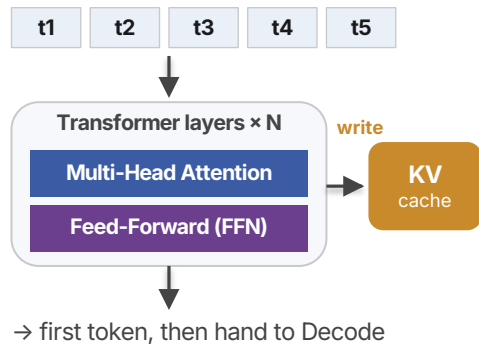
3D DRAM larger die (≤ 4 layers vs. 12–16 HBM): Better yield
SRAM-class bandwidth at ~1/10th HBM energy — short 3D IO, no sideways data movement, no PHY, no beachfront limit

Serving LLM Inference Workloads

LLM Inference

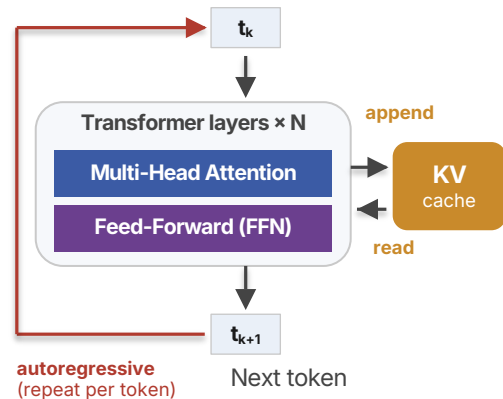
Prefill Stage

Prompt tokens (all, in parallel)

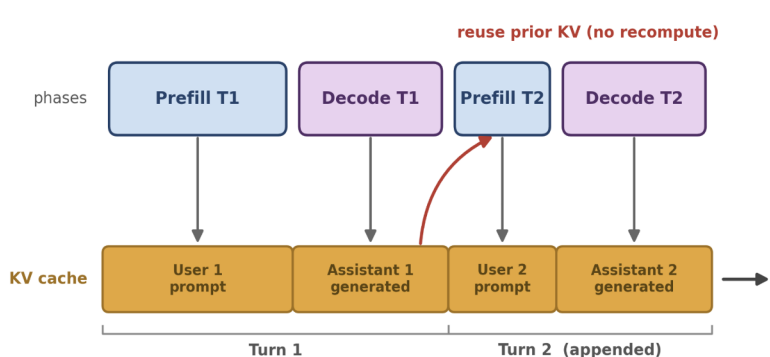


Decode Stage

Current token (1)



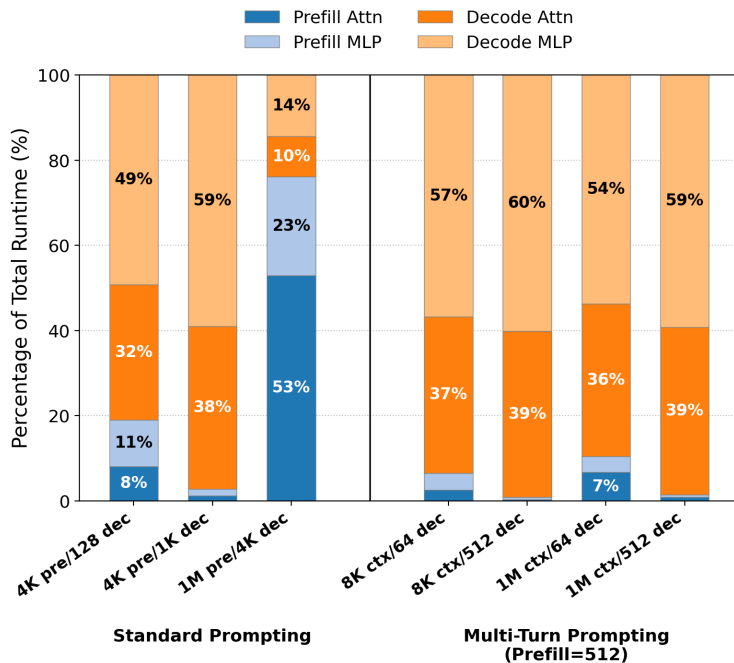
Agentic



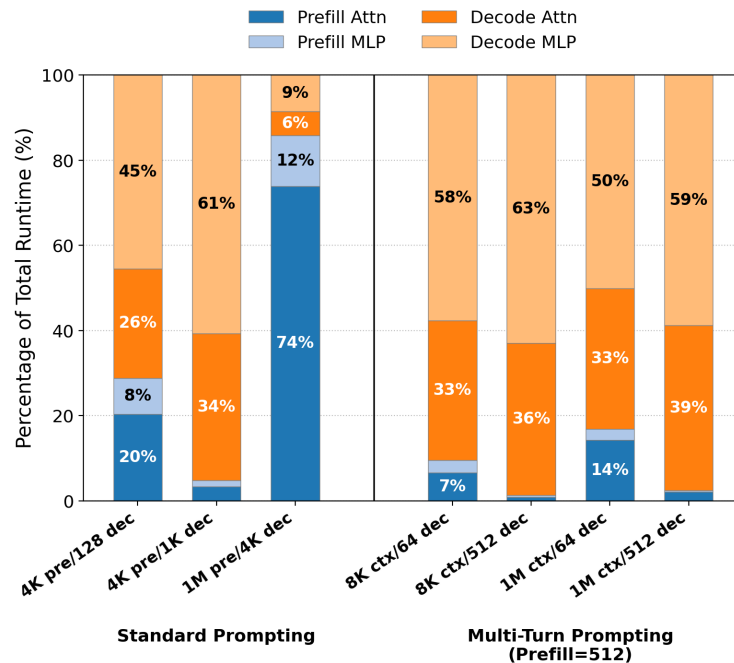
- Prefill Stage is **compute-throughput bound**
 - Large M dimension from processing multiple tokens at a time (irrespective of batch size)
- Decode Stage is typically **memory-bandwidth bound**
 - Small M dimension (small, modest batch sizes for low-latency scenarios)
 - Attention can become compute bound with high GQA + speculative decoding (independent of batch size)
 - MoE is memory-bandwidth bound even for modest batch sizes + speculative decoding

Majority of wall-clock inference time is spent in decode

**GLM 5.2 (MLA + DSA, 72-card) on Raptor:
prefill/decode runtime share by scenario**



**Kimi K3 (KDA + NoPE-MLA, 72-card) on Raptor:
prefill/decode runtime share by scenario**



Decode Phase (memory bound component) is a critical part of overall inference runtime.

1Hi 32GB 3D-DRAM: Frontier LLMs fit in one Raptor Rack

Sizing assumptions

Precision

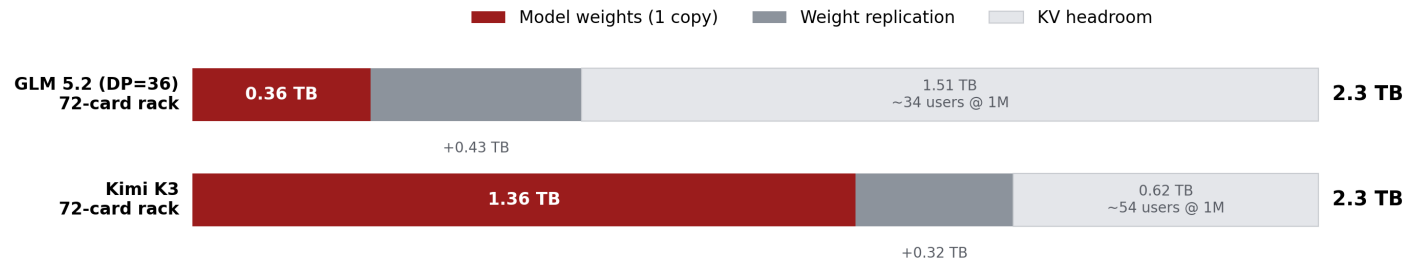
4-bit weights · 8-bit KV cache

Scale-up diameter

72 cards

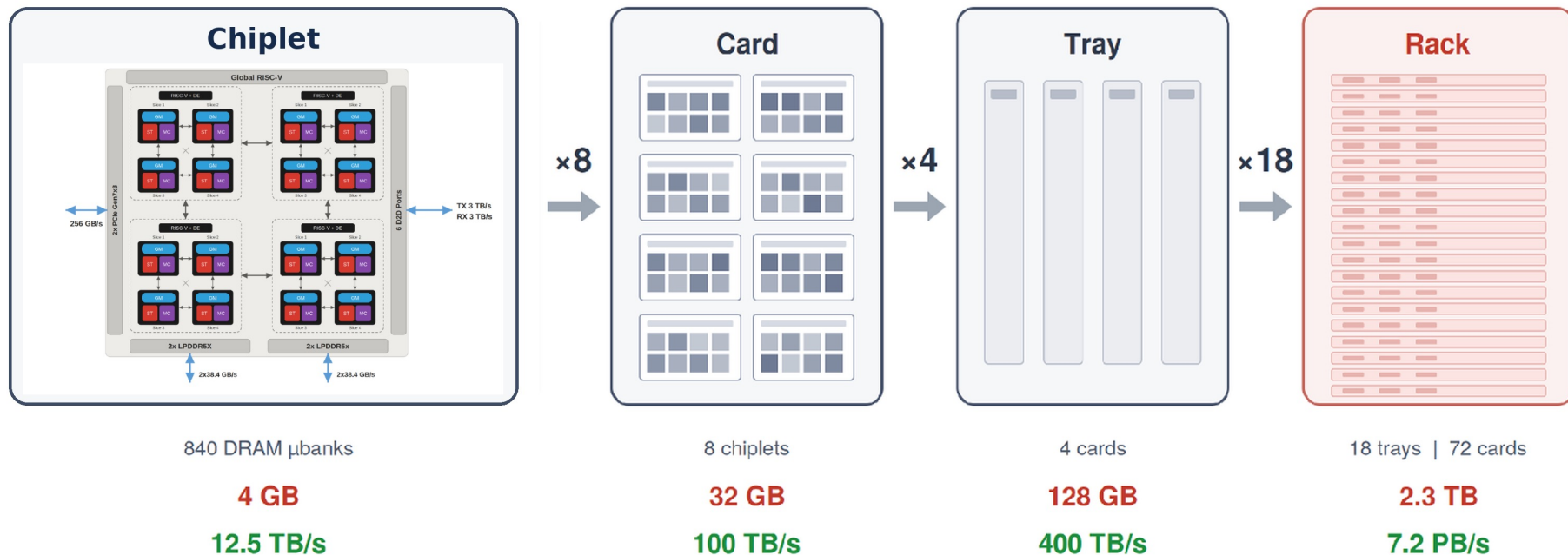
Model Details

Model	Parameters (MoE)	Context	KV / user
GLM 5.2	744B total · 40B active	1M	44.7 GB
Kimi K3	2.8T total · ~102B active	1M	11.7 GB



72-card scale-up at 32 GB/card hosts Kimi K3 at 1M context. Disaggregation & multi-rack extends beyond

From Raptor Chiplet to Rack



Building a 3D-DRAM Based Inference System

COMPLEX CO-DESIGN



1

Rethinking Memory Subsystem

Addressing memory bandwidth bottlenecks through innovative 3D-DRAM technology.

NEXT SUBSECTION



2

Rethinking Data Movement Fabric

Low latency is a must-have to avoid being cornered by **Amdahl's Law**.

- Topology
- Protocol
- Physical layer

THIS SECTION



3

Workload Mapping

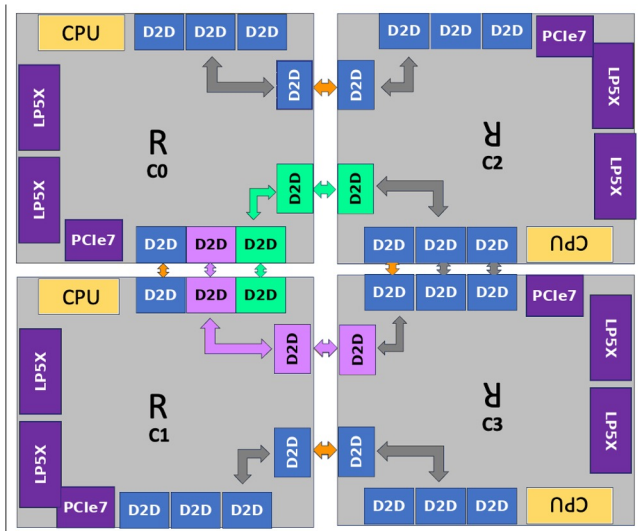
Sharding and collective communication algorithms.

Requires coordination of **8K virtual devices** over a hierarchical network vs. ~72 devices in a typical GPU system.

NOT COVERED

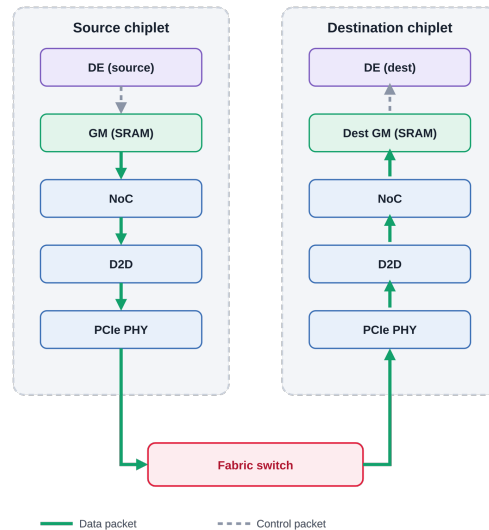
Low Latency Fabric (Intra-Card and Inter-Card)

TOPOLOGY



- **Full-mesh package** reduces intra-package latency.
 - Realized via “virtual” diagonals (green / magenta D2D).
 - Physical diagonals increase distance (package size, D2D power) and package layers.

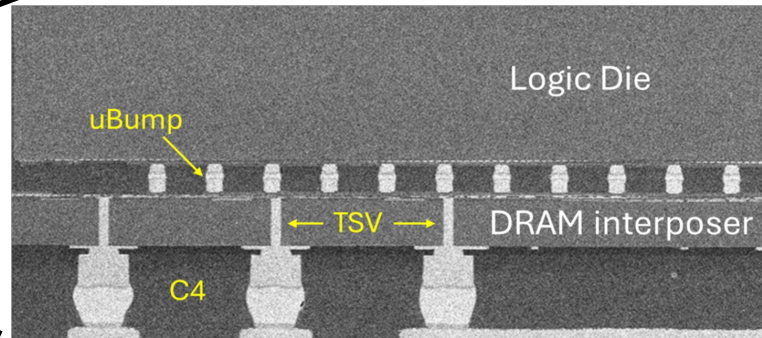
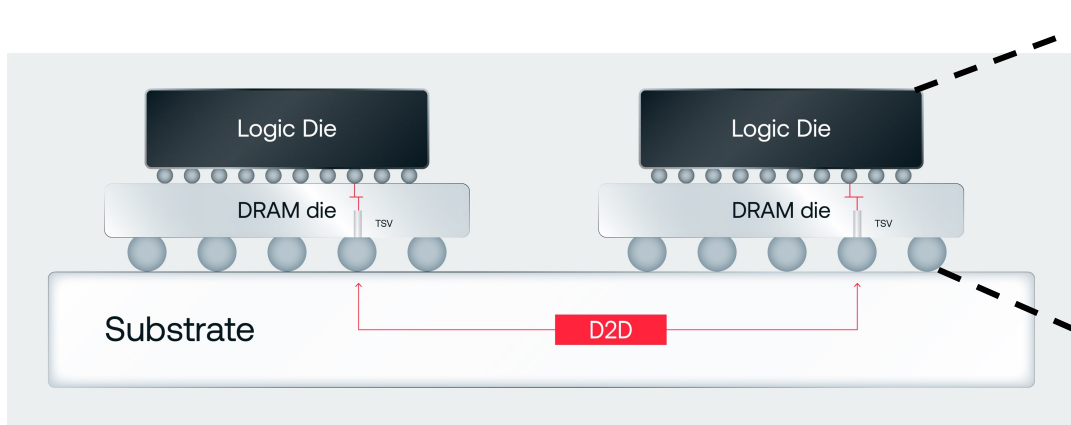
PROTOCOL



- **Push-style** end-to-end — intra-card and inter-card.
 - Device initiated.
 - Single-sided — source card doesn't receive an ACK.
 - Efficient support for multiple in-flight short-message transfers on a card.

3D-DRAM: The Opportunity

Raptor 3D-DRAM



- Top die: TSMC N4 logic
- Bottom die: 3D DRAM
- Integration: 36 μ Face to Face (F2F) stacking
- Proven low cost, high volume, high yield process

	HBM4	3D-DRAM
Topology	2.5D (beside)	F2F (below)
I/O energy	2–3 pJ/bit	0.37 pJ/bit
BW / card	18 TB/s	100+ TB/s
Capacity	192 GB	32 GB

5.6×

higher BW / card

5–8×

lower energy / bit

7×

denser I/O

The 3D-DRAM Integration Landscape

Hardware

TSV density vs. routing congestion trade-offs

Power delivery through 3D stack (IR drop, decap)

Signal integrity: crosstalk & SSN on wide parallel bus

★ I/O power: no burst structure $\Rightarrow$ no PHY DBI

μ Bump yield & alignment at 36 μ m pitch

Clock distribution & skew across die stack

Architecture

★ Bank-to-channel mapping: 840 banks $\neq$ 256 $\times$ 4

Weight & KV-cache tiling across banked memory

★ Redundancy without breaking channel symmetry

LPDDR5X as secondary tier: data placement policy

Compiler support for stream-aware access patterns

Runtime KV-cache alloc & eviction across banks

Co-Design

★ Thermal: $T_j=105^\circ\text{C}$
 $\Rightarrow$ 8 $\times$ faster refresh

ECC, DBI metadata, & scrub share bank rows

Refresh scheduling vs. compute pipeline bubbles

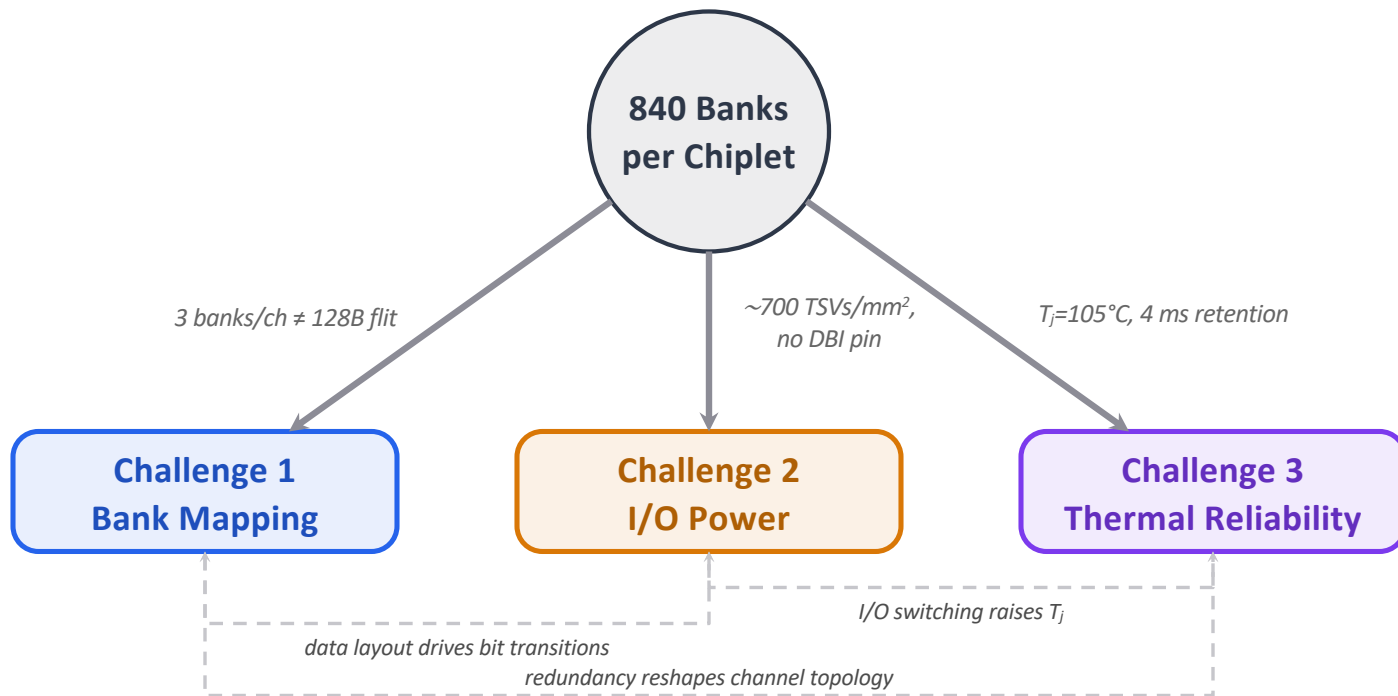
Known-good-die testing
 $\rightarrow$ post-package repair flow

Multi-high stacking: routing, power, thermals

Hybrid bonding roadmap: <2 μ m pitch, 8-high

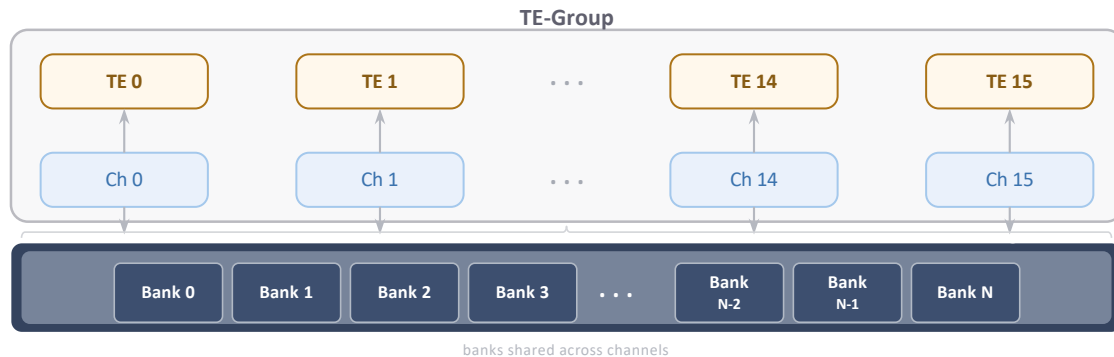
★ The challenges we discuss in this talk

These Challenges Are Entangled



A solution to any one challenge **constrains the design space of the other two.**

Challenge 1: The Bank-to-Channel Mapping Problem



The setup

- Each TE needs a **128 B flit** per access
- 16 ch $\times$ 16 TE-Groups = **256 ch/chiplet**
- Each bank delivers **32 B** per column access

The arithmetic

- $128 \text{ B} \div 32 \text{ B} = 4 \text{ banks/ch}$ *needed*
- DRAM die: **840 banks** (768 after 72 spares)
- $768 \div 256 = 3 \text{ banks/ch}$ *available*

Need 4 banks/channel $\Rightarrow$ have only 3 $\Rightarrow$ **the 128 B flit does not divide evenly**

Challenge 1: The Overfetch Dilemma



With 3 banks per channel:

- One channel access returns **96 B** (3 x 32 B)
- 128 B flit $\Rightarrow$ two accesses $\Rightarrow$ 192 B fetched
- **33% overfetch $\approx$ 33 TB/s wasted**

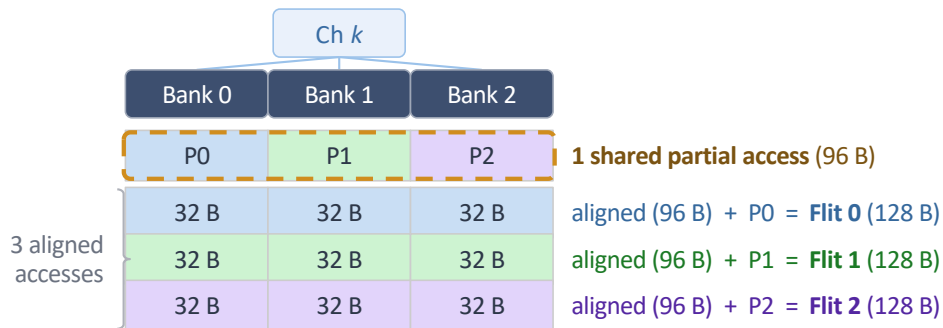
Naïve fix — column staggering:

- Stagger column indices to pack one flit per pair
- Requires a **192 B shifting buffer**
- Complicates timing closure and verification

How to deliver 128 B from 3×32 B banks **without**
wasting bandwidth or adding complex datapaths?

Solution: Stream Blocking

Challenge. A 128 B flit = 4×32 B, but each channel has only **3 banks** — a naïve fetch reads 192 B per 128 B flit ($\Rightarrow$ **33% overfetch**).



Puts the 4th 32 B (the *partial*) into one access *shared* by 3 flits — $4 \times 96 = 384$ B in = $3 \times 128 = 384$ B out $\Rightarrow$ **0% overfetch**.

Every column access is fully utilised — the **33 TB/s** lost to overfetch is reclaimed, with **no shifting network**.

Challenge 2: The I/O Power Wall

The power cost of 100 TB/s:

Measured I/O Power

$$P_{I/O} = 100 \text{ TB/s} \times 0.37 \text{ pJ/bit} = \mathbf{296 \text{ W}}$$

Why conventional DBI cannot help:

- HBM uses multi-cycle bursts (BL 4–16)
- PHY sees full burst $\Rightarrow$ decides on inversion
- DBI pin per byte lane signals choice

Our 3D-DRAM has **none of these**: single-cycle transfer, no burst, no sideband pin.

Interface comparison:

DDR / HBM	
Transfer	Multi-cycle burst
Lookahead	Full burst visible
DBI	Dedicated pin/lane
Our 3D-DRAM	
Transfer	Single-cycle 256-bit
Lookahead	None
DBI	No pin. No sideband.

DBI would save 20% power

Need to rethink interface.

How to achieve DBI **architecturally**?

Solution: Stream Flipping (Pinless DBI)

Challenge. I/O dominates: $100 \text{ TB/s} \times 0.37 \text{ pJ/bit} = \mathbf{296 \text{ W}}$, and conventional DBI needs a **sideband pin** the single-cycle 3D-DRAM link doesn't have.



⇒ **long** per-channel flit streams; each flit compared to the *previous* one.

(a) Without DBI

flit $i-1$:	0 0 ... 0
flit i :	1 1 ... 1
flit $i+1$:	0 0 ... 0

→ every bit toggles twice

(b) Stream flipping: invert flit i

0 0 ... 0	0 meta
0 0 ... 0 (was 1...1)	1
0 0 ... 0	0

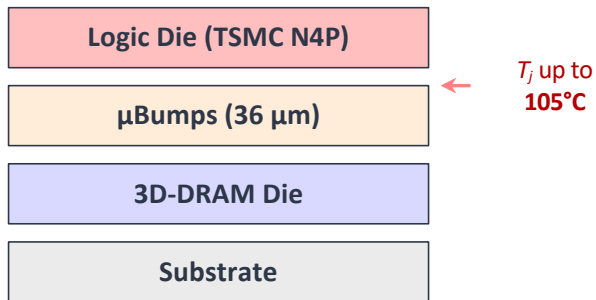
→ ≈ 0 toggles + 1 metadata bit / flit

Architectural DBI, no pin:

- **Write:** compare each flit to the previous → invert.
- **Read:** fetch the 1-bit tag; invert if set
- Tag co-located with ECC ⇒ **0.8%** overhead, **no PHY change** → **20% power savings.**

The full **20%** I/O power saving with **no pin** and **no PHY change.**

Challenge 3: DRAM Reliability at 105°C



Retention consequence

Standard: 32 ms at 85°C

At 105°C: drops to **4 ms**

⇒ **8x** more frequent refresh

Three compounding problems:

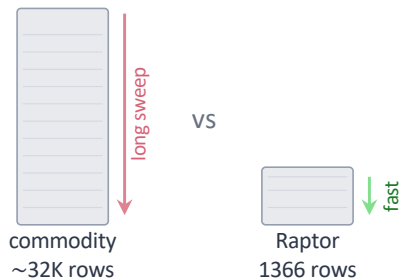
- 1. Yield** — 840 banks/die. Even a 1% fault rate risks losing entire channels. Discarding bonded dies is uneconomical.
- 2. Symmetry** — Disabling a faulty bank narrows its channel. Tensor engines expect *uniform* channel widths—one weak channel throttles the entire slice.
- 3. Error accumulation** — Higher T_j drives more soft errors. ECC, scrub, and refresh must all coexist without stalling throughput.

How to maintain full-bandwidth, symmetric channels when banks fail and refresh is **8x** nominal?

Solution: Thermal-Aware Refresh & ECC

Challenge. At **105°C**, retention falls 32 ms $\rightarrow$ 4 ms (**8x** more refresh), and bits still need correcting -- all *without losing significant bandwidth.*

Deep banking makes refresh cheap



16–32x fewer rows $\Rightarrow$ 4 ms refresh (8x more frequent) costs only **1.37%** $\rightarrow$ **~100 TB/s.**

Interleaved ECC, co-located with DBI



Last **8 of 124** columns: paired **[132,128]** Reed–Solomon + DBI bits, interleaved with the subarray.

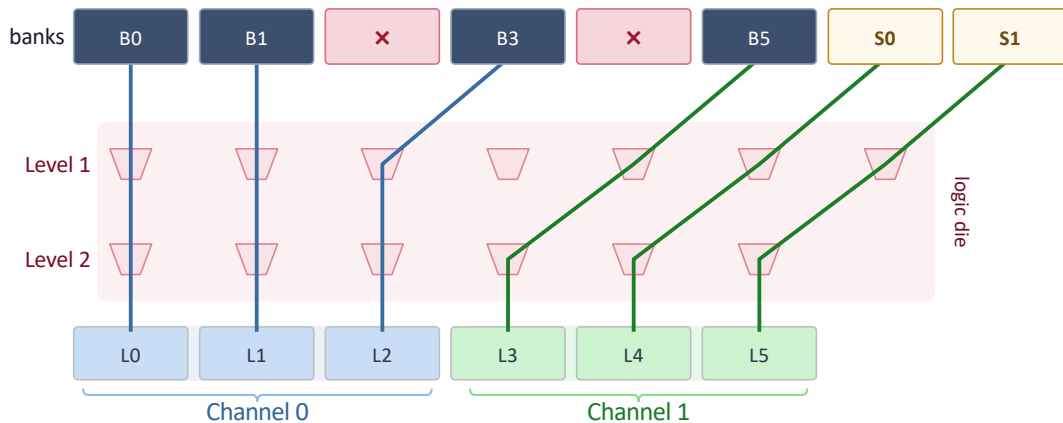
Read order:



Deep banking turns an 8x refresh penalty into only a **<1.4% bandwidth loss**

Solution: Bank Chaining

Challenge. The **72 spare banks** must cover faults *anywhere* on the die — without making channels asymmetric or adding long edge wires.



*Level 1 skips the first fault (B2); Level 2 skips the second (B4) — **M=2 physical mux levels tolerate 2 faults anywhere**, spares backfill, channels stay symmetric.*

The chiplet's **72 in-line redundant μ banks** absorb arbitrary faults -- symmetric channels, negligible routing cost.

Conclusions - 1

Our contribution

3D-DRAM puts bandwidth *and* capacity in one package → three co-designed solutions.

Raptor 3D-DRAM vs. HBM4 & Rubin R200 silicon-area basis

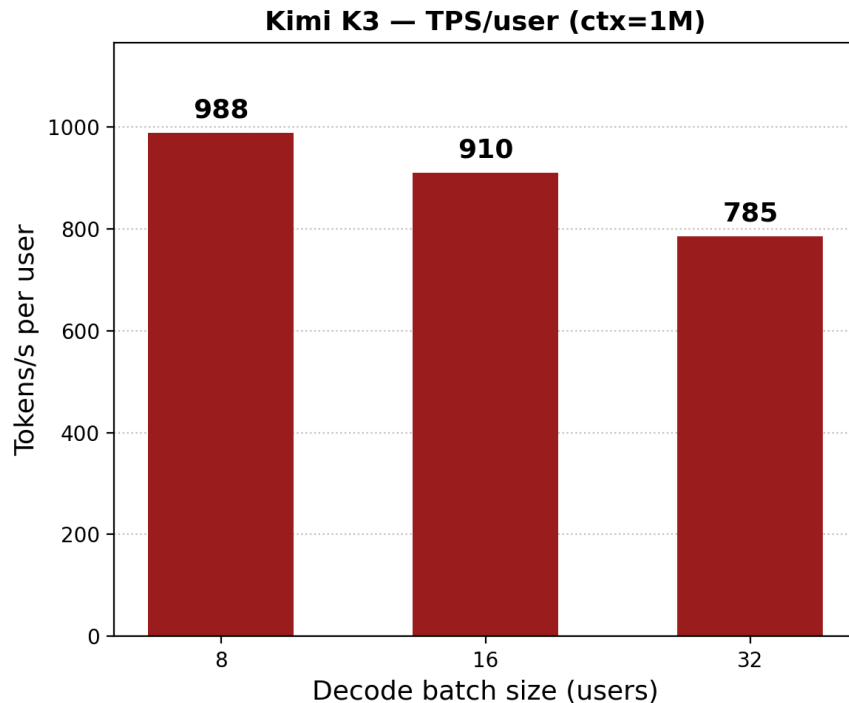
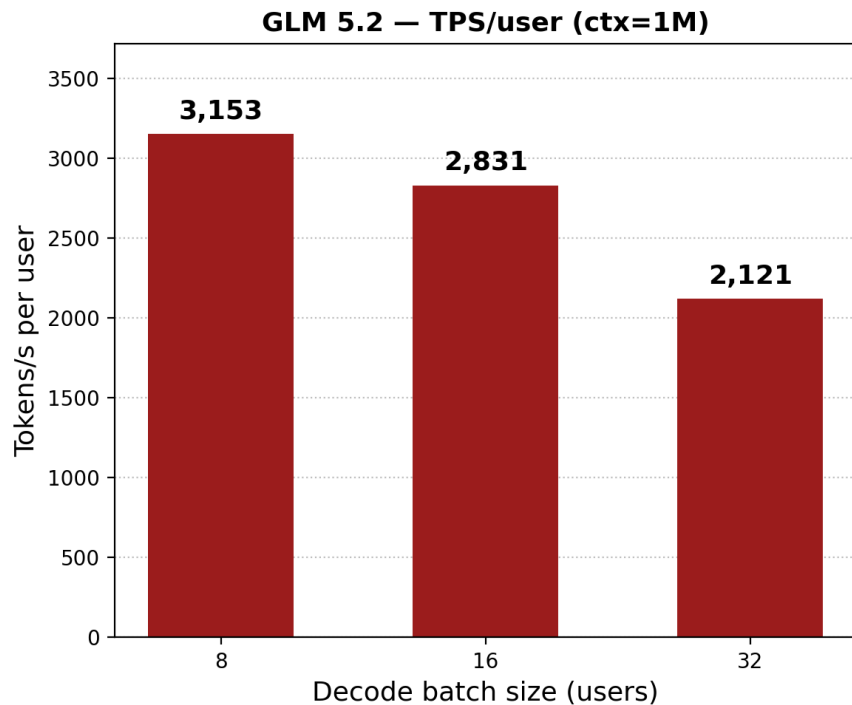
Metric	Raptor 3D-DRAM	HBM4 24 Gb	HBM4 32 Gb	Rubin R200
Capacity (MB/mm ²)	11.4	21.9	26.3	21.9
Bandwidth (GB/s/mm ²)	32.6	1.67	1.51	1.39
Power per GB/s (mW/GB/s)	2.96	40.0	40.0	40.0

Effective-BW basis (Raptor 83%, Rubin 85% util.). Power per GB/s: lower is better. Rubin R200 = 8× HBM4 24 Gb SoC.

≈20x the bandwidth per mm² and 13.5x better power per GB/s.

The memory wall is a technology problem. **3D-DRAM is the solution.**

Raptor sustains ~1000 TPS/User for serving 3T class model at 1M context !



Moderate per-card memory paired with high memory bandwidth wins Low Latency Inference

Early Silicon of Raptor

The First 3D-DRAM Accelerator for Generative Inference

Thank You

Early Silicon of Raptor: The First 3D-DRAM Accelerator for Generative Inference

Prashant J. Nair^{1,2}, Ramyad Hadidi¹, Subramani Ganesh¹, Sangamesh Kodge¹, Shubhankit Rathore¹, Neil Thanawala¹, Nikitha Reddy¹, Gyanesh Sahas¹, Vinayak Patankar¹, Anna Tirov¹, Nilesh Kurella¹, and Sudeep Bhoja¹

¹d-Matrix Inc. ²University of British Columbia

Abstract—Generative Inference is largely memory-bound. Autoregressive decoding dominates inference runtime and thus drives memory bandwidth and capacity demands. SRAM-based accelerators offer high bandwidth but limited capacity, while HBM DRAM provides capacity but is constrained by bandwidth and power. 3D-stacked logic-on-DRAM (3D-DRAM) helps close this gap, but integrating 3D-DRAM with accelerator logic introduces four challenges: (1) workload-aware mapping to exploit parallelism, (2) power optimization without burst-based data bus inversion (DBI), (3) resilience with high bank counts, and (4) thermal reliability at elevated junction temperatures.

This paper distills lessons from the early silicon of Raptor, the first commercial generative inference accelerator with 3D-stacked DRAM (3D-DRAM). This paper introduces four key architectural features to enable practical 3D-DRAM integration. First, stream-blocking maps KV-cache streams onto configurable 3D-DRAM channels, sustaining up to 100 TB/s per card while preserving bank-level parallelism. Second, pinless DBI on the single-cycle *plump* interface reduces memory-subsystem energy. Third, topology-preserving redundancy with thermal-aware refresh. Fourth, interleaved ECC for reliability at high temperatures. Across Llama-3.1 70B, DeepSeek-V3, Mistral 2, GPT-ONS, Whisper, and Canary, Raptor 3D-DRAM gives 4.71 $\times$ and 2.44 $\times$ higher throughput than with HBM and SRAM, respectively, while being less sensitive to network latency and bandwidth.

1. INTRODUCTION

Generative models now power interactive products such as search and assistants, as well as media tools, moving inference from offline batch jobs to always-on services with tight latency targets [2]. Unlike conventional compute-bound inference, serving large language and speech models is predominantly memory-bound. SRAM-centric designs have capacity limits, while HBM-based systems are constrained by bandwidth and I/O power. We show that a 3D-stacked logic-on-DRAM architecture delivers SRAM-like bandwidth and locality with HBM-class capacity at much lower energy, provided the accelerator is built around native 3D-DRAM integration rather than treating it as an add-on device. This paper presents the lessons learned and architectural insights from Raptor, the first-of-a-kind 3D-stacked DRAM-based accelerator.

Generative inference proceeds in two phases [18], [25]. The *prefill* phase is compute-bound and dominated by large matrix multiplications. The *decode* phase is autoregressive: tokens are generated sequentially, and each step issues multiple key-value (KV) cache reads and writes. Decode is therefore limited by the memory bandwidth [31], and this bottleneck grows as models and context lengths increase [47]. Once model weights and KV caches no longer fit within a single accelerator device, inference must be distributed across accelerators, adding

latency-heavy inter-card synchronization overheads [64]. In this regime, memory bandwidth and capacity become the primary limits on system performance [32].

Two metrics capture this performance. *Time Per Output Token* (TPOT) reflects session-level interactivity, while *tokens per second per accelerator card* (tok/s/card) measures throughput and utilization [92]. The memory system constrains both. For a Llama-class 70B-parameter model, weights occupy about 70GB at 8-bit precision, often exceeding a single card's capacity and forcing partitioning. During decode, each token attends to the accumulated KV cache across all layers. With 80 layers, grouped-query attention [3], roughly 8 KV heads, head dimension ~ 128 , and 8-bit storage, the KV cache grows by about 0.16GB per token per user [83]. This adds nearly 1.25GB at 8K, 5GB at 32K, and 20GB at 128K per user per session, directly limiting how many concurrent sessions a deployment can support on a given number of cards.

These characteristics and metrics determine the memory substrate. On a PCIe SKU, SRAM-centric accelerators provide about 150TB/s of on-die bandwidth but only a few GB of capacity [8], which does not scale to large model weights and KV caches. HBM4-based DRAM cards increase capacity to a few hundred GB but remain below 20TB/s of bandwidth and are limited by I/O power [1], [27], [75]. These constraints motivate Raptor, which vertically integrates 3D-DRAM with the accelerator logic. As shown in Figure 1, 3D-DRAM raises per-package bandwidth and capacity, enabling lower TPOT, higher tok/s/card, and reduced collectives.

While 3D-DRAM offers attractive bandwidth-per-GB scaling for generative inference, deploying it in a production accelerator exposes four system-level challenges. (i) The chiplet-based stack must present a clean, high-bandwidth interface to

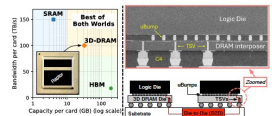


Fig. 1: The benefits of 3D-DRAM compared to SRAM and HBM. (a) 3D-DRAM offers a favorable performance-capacity trade-off. (b) 3D-DRAM connection of our 3D-DRAM integrated with the Raptor accelerator logic using face-to-face (F2F) stacking.